April 8, 2026

STOCKR

A STORE ITEM TRACKER

DEFINING THE SCOPE

OWEN FUGGER | OWENFUGGER@ICLOUD.COM

EXPERIENCES

The following projects informed the design decisions and technical approach behind the Item Tracker, each contributing practical experience in application structure, data management, and user interaction.

Online Support System - Java

A standalone terminal-based application designed to facilitate the submission, tracking, and resolution of support requests. This project established a foundation in building structured, menu-driven terminal interfaces and managing request lifecycles through distinct states.

Burnout AI, A Student Wellness Tracker - Python

A web application that helps students track daily habits, detect early burnout symptoms, and receive actionable recommendations for improvement. This project introduced full stack web development concepts and working with user-facing data in a browser environment.

Finance Tracker – C

A command-line application that allows users to register, log in, and manage personal income and expenses. Data is stored persistently using file operations, providing experience in user authentication, structured data storage, and building reliable CLI workflows in a low-level language.

PROJECT CONCEPT

HIGH LEVEL DESCRIPTION

Many small and independent stores rely on outdated or informal methods to track their inventory, orders, and sales. Spreadsheets become difficult to maintain, paper logs are prone to error, and off-the-shelf solutions are often overcomplicated or expensive for smaller operations. The Item Tracker addresses this gap by providing a lightweight, structured inventory management system built specifically for clarity and ease of use.

The system allows store managers to maintain a complete record of their inventory, including item descriptions, pricing, stock levels, storage locations, and order status. Sales are tracked and reported across daily, weekly, monthly, and yearly timeframes, giving management a clear picture of business performance at any point in time. Orders can be created, monitored, and closed out within the same system, reducing the need for separate tools or processes.

Access to the system is divided into two roles. The management account has full control over the inventory, including adding and removing items, updating stock levels, and managing orders. The customer-facing view is read-only, allowing customers or floor staff to look up product availability and details without the risk of accidental or unauthorized changes.

The project is built in three phases. The first phase delivers a fully functional terminal-based application, prioritizing sound data logic, a clean database structure, and reliable core functionality before any interface work begins. The second phase introduces a REST API built with FastAPI, tested in isolation before any frontend is connected. This ensures the backend logic is verified independently and working correctly. The

third phase extends the system to a locally hosted web interface, connecting a React frontend to the established API. This phased approach ensures each layer of the stack is solid before the next is introduced.

TARGET USERS

The Item Tracker is designed to serve four distinct user groups, each with different needs and levels of system access.

Store owners require a high-level view of business performance. The sales reporting and inventory valuation features give owners the data needed to make informed decisions about purchasing, pricing, and storage without having to dig through physical records or disconnected spreadsheets.

Managers are the primary operators of the system. They are responsible for maintaining accurate inventory records, processing orders, and ensuring stock levels reflect the current state of the store. The management account grants them full access to all features required to carry out these responsibilities.

Workers interact with the system on a day-to-day basis, using it to look up item locations, check stock levels, and confirm whether products are in active inventory or storage. Depending on store policy, workers may operate under the management role or a more restricted variation of it.

Customers are given a read-only view of the inventory, allowing them to check product availability and details independently. This reduces the need for staff to field routine stock inquiries and gives customers a straightforward way to find what they are looking for.

MAIN FEATURES

ITEM MANAGEMENT

The system must allow management to add and remove items from the inventory. Each item record contains a description, physical location, price, current stock level, and a flag indicating whether the item is in active stock or placed in storage.

INVENTORY VIEW

The system must provide a complete view of all items currently in stock, including individual item details and a calculated total dollar value of all merchandise on hand. Items held in storage must be viewable separately from active inventory.

SALES REPORTING

The system must track and display sales data across four timeframes: daily, weekly, monthly, and yearly. This gives management the ability to monitor business performance and identify trends over time.

ORDER MANAGEMENT

The system must support the creation and cancellation of orders for restocking items. Orders in progress and completed orders must each be viewable as separate lists, giving management a clear picture of outstanding and fulfilled supply needs.

ACCESS CONTROL

The system must support two user roles. The management role has full read and write access across all features. The customer role is restricted to read-only access, limited to browsing current inventory and item details.

BASIC REQUIREMENTS

ITEM MANAGEMENT

- Add new items to inventory
- Delete items from inventory
- Each item must contain:
    - Description
    - Location
    - Price
    - Amount in stock
    - Storage status

INVENTORY VIEW

- View all items in stock
- View individual item details
- View total dollar value of all merchandise
- View items in storage separately

SALES REPORTING

- View sales by:
    - Daily
    - Weekly
    - Monthly
    - Yearly

ORDER MANAGEMENT

- Add new orders

- Cancel existing orders
- View orders in progress
- View completed orders

ACCESS CONTROL

- Management account: full read and write access
- Customer account: read only access to inventory

SUCCESS SCENARIOS

A customer would like to know the location of an item in the store.

A customer enters the store and is looking for a specific product but is unsure where it is located. The customer accesses the system through the customer-facing view and searches for the item by name. The system returns the item record, displaying the product description, current stock level, and its location within the store. The customer is able to find the item without requiring assistance from a staff member.

A manager would like to see the profits for the day.

A store manager wants to review the store's financial performance at the end of the business day. The manager logs into the management account and navigates to the sales reporting section. The system displays a summary of all sales recorded for the current day, including a total revenue figure. The manager can assess the day's performance quickly and without consulting any external records.

TECH STACK

PHASE 1 – TERMINAL

Tool	Role
Python	Core language
SQLite	Data Base
SQLAlchemy	Communicates with Data Base
Rich	Clean terminal outputs
Typer	CLI commands and menus

PHASE 2 – API

Tool	Role
Python	Core language
FastAPI	Backend web API
SQLite	Data Base
SQLAlchemy	Communicates with Data Base
Postman	API testing tool

PHASE 3 - WEB

Tool	Role
Python	Core language
FastAPI	Backend web API
SQLite	Data Base
SQLAlchemy	Communicates with Data Base
React + Vite	Frontend UI
Tailwind CSS	Styling
Postman	API testing tool

LIMITATIONS

The Item Tracker in its current form is designed as a single-user, locally hosted application. It does not support simultaneous access from multiple users and is not intended for deployment on a public-facing server. The system relies on SQLite, which is well suited for local use but is not designed to handle concurrent write operations on a scale. Authentication in Phase 1 is basic by design, providing role separation without the security requirements of a production environment. The system does not currently support barcode scanning, automated reordering, or integration with third party point of sale systems.

FUTURE CONSISDERATIONS

As the project matures beyond its initial two phases, several directions are worth exploring. Migrating from SQLite to PostgreSQL would allow the system to support multiple simultaneous users and prepare it for potential cloud deployment. A third access tier for workers, separate from the full management account, could provide more granular control over what staff can view and edit. Barcode scanning integration would reduce manual data entry and improve accuracy at the point of receiving stock. Automated low stock alerts and reorder suggestions based on sales trends are also natural extensions of the reporting features already planned. These are not immediate priorities but represent a logical path forward as the system grows.

GLOSSARY OF TERMS

Item Tracker: A structured inventory management system that allows store staff to monitor stock levels, track sales, manage orders, and control access based on user role. The term refers specifically to this application and its associated database and interface.

Merchandise: The physical goods stored, tracked, and sold through the system. In the context of the Item Tracker, merchandise refers to any item entered into the inventory, whether on the active floor, held in storage, or currently on order.

Phase: A defined stage of development in the Item Tracker project. Phase 1 delivers a fully functional terminal-based application. Phase 2 introduces a REST API layer built and tested independently. Phase 3 connects a React frontend to complete the full web interface. Each phase must be complete and stable before the next begins.

Database: A structured collection of data that the Item Tracker uses to persistently store all inventory records, sales history, and order information. The system uses SQLite as its database, managed through SQLAlchemy.

CLI (Command-Line Interface): The text-based interface used in Phase 1 of the Item Tracker. Users interact with the system by navigating menus and entering commands in the terminal rather than through a graphical or web-based interface. Built using Typer and Rich.

API (Application Programming Interface): In the context of Phase 2, the API is the layer built with FastAPI that sits between the database and the React frontend. It receives requests from the browser, performs the appropriate database operation, and returns the result to the user interface.

ORM (Object Relational Mapper): A programming technique that allows a developer to interact with a database using the programming language of their choice rather than writing raw SQL queries directly. SQLAlchemy is the ORM used in this project, translating Python code into database operations automatically.

Frontend: The part of the system that the user directly interacts with. In Phase 2 of the Item Tracker, the frontend is the web interface built with React that runs in the browser and displays inventory, sales, and order information to the user.

Backend: The part of the system that runs behind the scenes. In Phase 2, the backend is the FastAPI layer that receives requests from the frontend, communicates with the database, and returns the appropriate data back to the user interface.

Inventory: The complete collection of items currently tracked within the system. This includes items on the active floor, items held in storage, and items currently on order. The inventory is the central data set that all features of the Item Tracker are built around.

Stock: The quantity of a specific item currently available on the active floor. Stock is distinct from storage, which holds items that are present in the building but not yet available for sale.

Read/Write Access: The level of permission granted to a user within the system. Read access allows a user to view information without making changes. Write access allows a user to add, edit, or delete records. Management accounts have both read and write access while customer accounts are limited to read access only.

Postman: A professional API testing application used during Phase 2 and Phase 3 of the Item Tracker project. Postman allows a developer to manually send HTTP requests to API endpoints and inspect the responses without needing a frontend interface. It is used to verify that the backend is functioning correctly in isolation before any frontend code is connected.